Relational Algebra.

- In DBMS, Relational Algebra is a formal, procedural querry lang. that uses operators to perform op^{n}s on relations to produce new relations as o/p.
- It forms the theoritical foundation for SQL and allows users to retrieve and manipulate data in a relational d/b by defining queries in a precise, mathematical way.

Types of Relational op^{n}s.

- Select (σ)
- Project (π)
- Union ($\cup$)
- Rename (ρ, $\leftarrow$)
- Intersection ($\cap$)
- Difference ($-$)
- Cartisian ($\times$)
- Join ($\bowtie$)
- Division ($\div$)

1.) Select (σ) :- Produce table containing subset of rows of arguments table satisfying condition.

condition (Relation)

eg:

Id	name	hobby	address
1	John	Stamp	123 A
1	John	coin	123 A
2	Mary	Stamp	7 lake.
3	Ravi	Coin	5 Road

(Person)

$\sigma_{\text{hobby='stamp'}}$ (Person).

Id	name	hobby	address
1.	John	Stamp	123 A
2.	Mary	Stamp	7 lake

2.) Project (π) :- Produce table containing subset of col^m of argument table.

$\pi_{\text{(attribute list)}}$ (Relation)

Find name, address of all persons

$\Pi_{name, address}$ (Person).

Name	address
John	123A
Mary	7 lake
Ravi	5 Road

(This also eliminates duplicate values).

eg: Find the names of person whose hobby is stamp or coin.

$\Pi_{name}(\sigma_{hobby = 'stamp' \text{ or } hobby = coin}(Person))$.

3.) <u>Renaming</u>: We can rename the relation that hold the intermediate result using assignment operator.

eg: $\Pi_{name, salary}(\sigma_{dno=5}(Emp))$

or

$D_5 Emp \leftarrow \sigma_{dno=5}(Emp)$

$Result \leftarrow \Pi_{name, salary}(D_5 Emp)$.

We can also use this techiques to rename the attribute in the intermediate and result relⁿ.

eg: $R(E\text{-}name, E\text{-}sal) \leftarrow \Pi_{name, salary}(D_5 Emp)$.

We can also use ρ symbol for renaming relation name or attribute name. or both.

$\rho_{S(B_1, B_2, .. B_n)}(R)$.

ρ → symbol of renaming
S → new table name.
$B_1, B_2 .. B_n$ → new colⁿ name.

$\rho_S(R)$. or $\rho_{(B_1, B_2, .. B_n)}(R)$

Union op^n ($\cup$): It contain all tuples that are either in relation R or S or in both. It eliminate duplicate tuples.

$$\boxed{R \cup S}$$

eg: Find the name of all customer who have either an acc or loan or both.

c-name	acc-no

depositor

c-name	loan-no.

Borrower

$$\boxed{\Pi_{c\text{-}name}(\text{Depositor}) \cup \Pi_{c\text{-}name}(\text{Borrower})}$$

If relations are union compatible then only they can combined using, $\cup$, $\cap$, $-$

Union Compatibility

- → Both have same no. of col^m
- → Name of attributes are same in both.
- → Attributes with same name in both rel^n have the same domain.

Example: person (ssn, name, add, hobby).
professor (id, name, office, phone) are not union compatible.

But Π_{name} (Person) and Π_{name} (professor) are union compatible.

Intersection compatibility ($\cap$):-

R & S are union compatible.

$$\boxed{R \cap S}$$

Set Difference Operator :- Find tuples that one relation but are not in ~~of~~

They are taken b/w compatible relations.

$R - S \neq S - R$.

Cartesian Product Operator (x) :

C-name	L-no
Ashish	L1
Ram	L2
Seeta	L3

Borrow.

Lno	Branch	amount
L1	Pune	400
L2	Noida	600

Loan

Borrow x Loan

C-name	L-No	L NO	Branch	Amount
Ashish	L1	L1	Pune	400
Ashish	L1	L2	Noida	600
Ram	L2	L1	Pune	400
Ram	L2	L2	Noida	600
Seeta	L3	L1	Pune	400
Seeta	L3	L2	Noida	600

If relations have n_1 and n_2 tuples respectively. then resultant cartesian table will have $n_1 \times n_2$ tuples.

eg: loan [L-no | branch_name | amount]

borrow [customer_name | loan_no.]

suppose we want to select the name of customer who have loan in ABC Branch.

$\Pi_{customer_name} (\sigma_{borrow.loanno = loan.lno} (\sigma_{branch.name = ABC} (Borrow \times loan))$

eg: Branch (B-name, B-city, assets)
Customer (C-name, C-street, C-city)
Account (A-no, B-name, Bal)
Loan (L-no, B-name, Amt)
Depositor (C-name, A-no)
Borrower (C-name, L-no)

① Find all loans of over $1200

Sol^n $\sigma_{Amt > 1200}$ (loan)

② Find customer name, acc-no. of all depositors.

Sol^n $\Pi_{C\text{-}name,\ A\text{-}no}$ (Depositor)

③ Find all acc-no whose balance is less than $5000

Sol^n $\Pi_{A\text{-}no}$ ($\sigma_{Bal < 5000}$ (account))

④ Find the loan no. for each loan of an amt greater than $1200

Sol^n $\Pi_{L\text{-}no.}$ ($\sigma_{Amt > 1200}$ (loan))

eg: of Set Diff:

Student

Fn	Ln
Susan	Yaa
Ramesh	Shah
Johny	Kohli.
Barbara	Jones.
Amy	Ford
Jimmy	Wang.
Ernest	Gilbert

Instructor

F.name	L-name.
John	Smith
Ricardo	Browne.
Susan	Yaa.
Francis	John
Ramesh	Shah.

S − I =

Johny	Kohli
Barbara	Jones.
Amy	Ford
Jimmy	Wang.
Ernest	Gilbert

Division operator (÷)

Division can be applied (A ÷ B) if and only if

- Attributes of B is proper subset of attributes of A
- The relation returned by division operator will have attributes (All attributes of A − all attributes of B).
- It will return those tuples from A which are associated to every tuple in B.

Student_sports.

Roll No	Sports
1	Badminton
2	Cricket
3	Badminton
4	Badminton

All_sports.

Sports
Badminton
cricket.

Result:

Roll NO
2

- This is valid as all_sports is a subset of Student_sports
- Resulting attributes {Roll No, Sports} − {Sports} = Roll NO.
- only Roll NO 2 is associated with all tuples of B.

Join :- A join is a query that combines rows from two or more tables.

Join Condition:

Most join queries contains atleast one join condition which compare two col^n each from different table. To execute a join, oracle engine combines pair of rows each containing one row from each table for which join condition evaluates to be 'true'.

Types of Joins :-

Equi Joins :- A join with a join cond having an equality operator.

Self Join :- A join of a table to itself.

Inner Join :- (Simple join) It is a join of two tables that returns only those rows that satisfy the condition.

eg:

id	name
123	A
124	B
125	C
126	D.

Student

id	code	Phone.
123	W	1
124	X	2
124	Y	3
126	Z	4

Enrol

Student ⋈ Enrol
Student.id = Enrol.id

col^n names in both tables can be diff.

id	name	id	code	Phone.
123	A	123	W	1
124	B	124	X	2
124	B	124	Y	3
126	D	126	Z	4

Inner Join.

There may be any condition other than equality

eg:

Emp	Street	Budget
Ram	X	100
Shyam	Y	150
Ravi	Z	80

Buyer

Property	Price
A	85
B	125
C	175

Property

Emp	Street	Budget	Property	Price
Ram	X	100	A	85
Shyam	Y	150	A	85
Shyam	Y	150	B	125

Buyer ⋈ Property

Price <= Budget

~~Outer Join~~:

~~SQL: Select * from.~~

Outer Join : It returns all rows that satisfy the join cond^n and also return some or all those rows from one table for which no rows from the other satisfy the join condition.

They are of 3 types:

i) Left outer join (⟕): Returns all rows from left table, that did not match any tuple in the right table. Pads the tuple with NULL value for all the attributes from right relation.

emp	street	city
A	X	Mum
B	Y	Kol
C	Z	Delhi
D	W	Hyd

(Emp)

emp	branch	Salary
A	CS	1000
B	IT	2000
E	EN	3000
D	EC	5000

(work)

Emp	Street	city	emp	branch	Salary
A	X	Mum	A	CS	1000
B	Y	Kol	B	IT	2000
D	W	Hyd	D	EC	5000
C	Z	Delhi	NULL	NULL	NULL

(emp ⟕ work)
emp.emp = work.emp.

ii.) Right outer join (⟖) :- Returns all rows from right table.

emp	Street	city	emp	branch	Salary
A	X	Mum	A	CS	1000
B	Y	Kol	B	IT	2000
D	W	Hyd	D	EC	5000
NULL	NULL	NULL	E	EN	3000

emp ⟖ work.
emp.emp = work.emp.

iii.) Full outer join (⟗) :- Extra rows of both tables are then

Natural Join :- It is based on all columns in two tables that have the same name. It select rows from two tables that have equal values in the relevant colⁿ.

Student ⋈ Enrol.

→ In this no need to write join condⁿ because it always works on colⁿ with same name & values.

→ It displays the colⁿ only once.

id	name	code	ph
123	A	W	1
124	B	X	2
124	B	Y	3
126	D.	Z	4

It is subset of equijoin.

Practice Questions:

Q-1 Student (ssn, name, add, major)
Course (code, title)
Registered (ssn, code)

a.) List the code of courses in which atleast one student is registered

Ans: Π_{code} (Registered)

b.) List the title of registered courses.

Π_{title} (course $\bowtie$ Registered)

c.) List the code of courses for which no student is registered.

$T \leftarrow \Pi_{code}(course) - \Pi_{code}(Registered)$

d.) The titles of courses for which no student is registered.

$\Pi_{title}(T \bowtie course)$

e.) Name of students and title of courses they register to.

$\Pi_{name, title}(S \bowtie R \bowtie C)$

f.) ssn of student who are registered for Both DBMS and DAA

$X \leftarrow \Pi_{ssn}(\sigma_{title=DAA}(S \bowtie R \bowtie C)) \cap \Pi_{ssn}(\sigma_{title=DBMS}(S \bowtie R \bowtie C))$

g.) Name of student who are registered for both DBMS and DAA.

$$\pi_{name}(X \bowtie Student)$$

h.) List of courses in which all students are registered.

$$\pi_{code,ssn}(Registered) / \pi_{ssn}(Student)$$

i.) List of courses in which all 'ECMP' major students are registered

$$\pi_{code,ssn}(Registered) / \pi_{ssn}(\sigma_{major=ECMP}(Student))$$

Q. (S) Sailor (sid, sname, rating, age).
(B) Boat (bid, bname, color).
(R) Reserved (sid, bid, date).

1.) Find color of boat reserved by Albert

$$\pi_{color}(\sigma_{sname = 'Albert'}(S \bowtie R \bowtie B))$$

2.) Find all Id's of sailors who have rating atleast 8 or reserved boat 103.

$$\pi_{sid}(\sigma_{rating >= 8}(S)) \cup \pi_{sid}(\sigma_{bid = 103}(R))$$

3.) Find the name of sailor who have reserved Red Boat.

$$\pi_{sname}(S \bowtie (R \bowtie \sigma_{color = 'Red'}(B)))$$

4.) Find the name of sailor who have not reserved Red boat.

$$\pi_{sname}((\pi_{sid}(S) - \pi_{sid}(\sigma_{color = 'Red'}(B) \bowtie R)) \bowtie S)$$

5.) Find the name of sailor who have reserved all boats.

$$\pi_{sname}((\pi_{sid, bid}(R) \div \pi_{bid}(B)) \bowtie S)$$

6. Find the id of sailor whose age > 20 and who have not reserved red boat.

$$\pi_{sid}(\sigma_{age > 20}(S)) - \pi_{sid}(\sigma_{color = 'Red'}(B) \bowtie R)$$

(7) Find the name of sailor who have atleast two boat. (We have to relate R to itself).

$$\pi_{sname}\left(\left(\sigma_{R.sid = R2.sid \wedge R.bid <> R2.bid}\left(R \bowtie \rho(R_2, R)\right)\right) \bowtie S\right)$$

explaination.

sid	bid
1	A
2	B
3	A
1	C

sid	bid
1	A
2	B
3	A
1	C

$\Rightarrow$

sid	bid	sid	bid
1	A	1	A
1	A	1	C
2	B	2	B
3	A	3	A
1	C	1	C

eg: E emp (eid, ename, estreet, ecity)
W work (eid, cid, salary)
L loc (ccode, cname, ccity)

1. Find the name of employe who work in company TCS

$$T \leftarrow \Pi_{ename}(\sigma_{cname='TCS'}(E \bowtie W \bowtie L))$$

2.) Find the name of all employee who work for a company localid in 'lko'.

$$\Pi_{ename}(\sigma_{ccity='lko'}(E \bowtie W \bowtie L))$$

3.) Find the name of employee who do not work for TCS

$$\Pi_{ename}(E) - T$$

4.) Find the name of emp who work for the company localid at the same city in which they live.

$$\Pi_{ename}(\sigma_{ecity=ccity}(E \bowtie W \bowtie L))$$

5.) Find the address of an employee who gets the salary more than 1000/- and working in TCS.

$$\Pi_{estreet, e\text{-}city}(\sigma_{cname='TCS' \wedge salary>1000}(E \bowtie W \bowtie L))$$